



CMPT 295

Unit - Instruction Set Architecture

Lecture 24 – Introduction to Instruction Set Architecture (ISA)

Today's Menu

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Example of an instruction set: MIPS
 - Create our own instruction sets
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Staged execution of machine instructions
 - Pipelined execution of machine instructions + Hazards

Reference

- [Computer Organization and Design](#), 5th Edition, 2014 by David A. Patterson and John L. Hennessy
 - See [Resources](#) for a link to an online version
 - Chapter 2 – Instructions: Language of the Computer
 - Chapter 4 – The processor
- Chapter 4 of our textbook
 - We will not make use of this chapter very much!

The Big Picture – Above the hood

C code

```
short abs( short aNumber ) {  
  
    short result = 0;  
  
    if ( aNumber > 0 ) result = aNumber;  
    else result = -aNumber;  
  
    return result;  
}
```

Assembly code

```
.globl  abs  
abs:  
    movl    %edi, %eax  
    testw   %di, %di  
    jle     .L3  
.L2:  
    ret  
.L3:  
    negl     %eax  
    jmp     .L2
```

Machine code

```
1111100010001001  
111111111000010101100110  
0000001001111110  
1100001111110011  
1101100011110111  
1111101011101011
```

C program (.c) → abs.c

Preprocessor
Preprocessed Source → abs.i

C Compiler

Assembly program (.s)

→ abs.s

Assembler

Object (.o) → abs.o

Linker
Executable → abs

ISA - Instruction Set Architecture

Loader

Computer executes it

CPU

Memory

What Is an Instruction Set Architecture?

An Instruction Set Architecture (ISA) is part of the abstract model of a computer that defines how the CPU is controlled by the software. The ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done.

The ISA provides the only way through which a user is able to interact with the hardware. It can be viewed as a programmer's manual because it's the portion of the machine that's visible to the assembly language programmer, the compiler writer, and the application programmer.

What Is an Instruction Set Architecture? cont'd

The ISA defines the supported data types, the registers, how the hardware manages main memory, key features (such as virtual memory), which instructions a microprocessor can execute and the input/output model of multiple ISA implementations. The ISA can be extended by adding instructions or other capabilities, or by adding support for larger addresses and data values.

Like the
SSE

Why the ISA Is Important?

Understanding what the instruction set can do and how the compiler makes use of those instructions can help developers write more efficient code. It can also help them understand the output of the compiler which can be useful for debugging.

We say that a microprocessor *implements* an ISA.

The Big Picture - Under the hood!

MIPS Microprocessor datapath

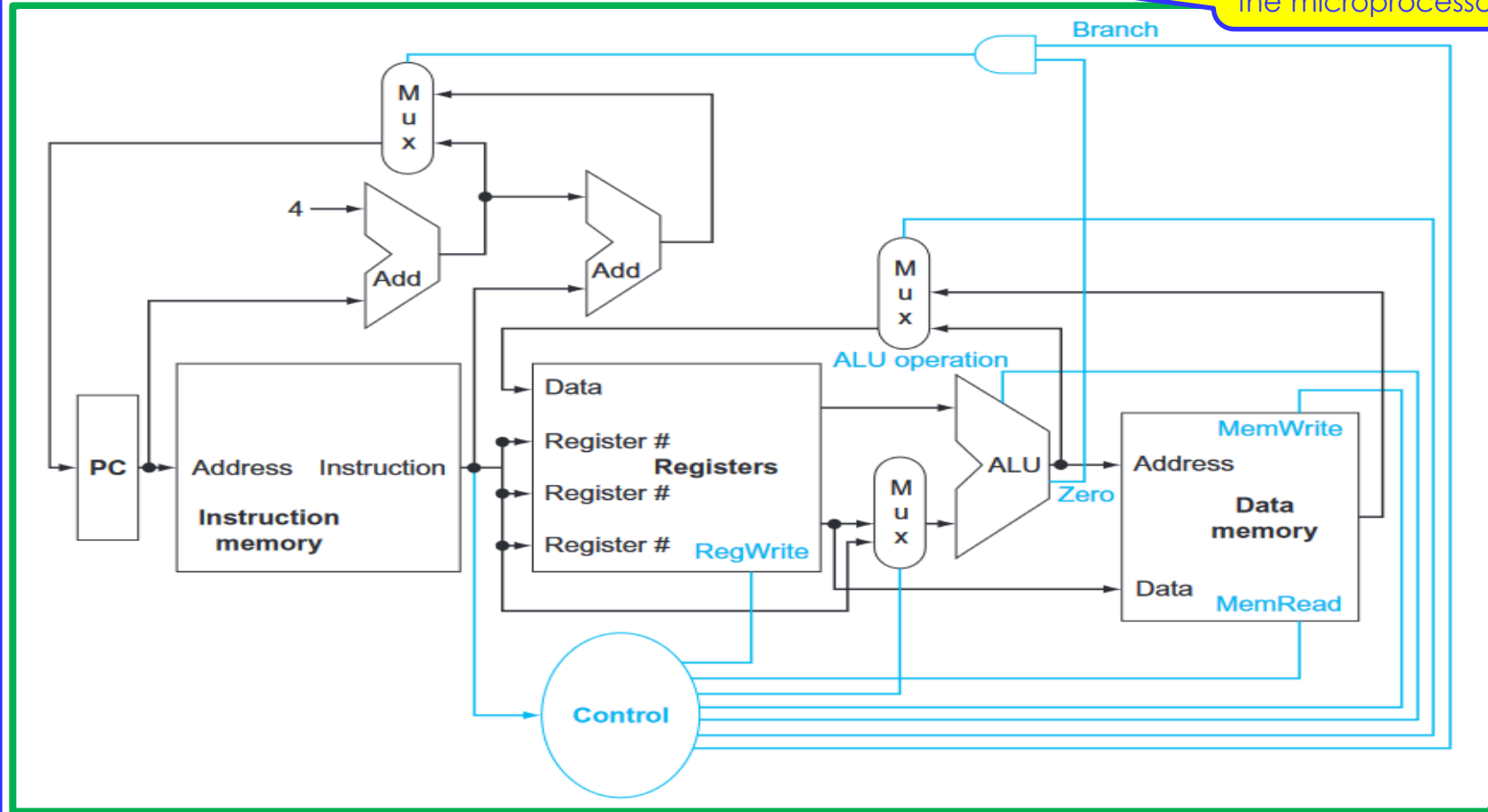
Abstract representation of h/w components making up the microprocessor

Wikipedia says:

A **datapath** is a collection of functional (h/w) units such as

- ALU (perform data operations),
- registers, and
- buses (allow data to flow between them).

Along with the control unit, the **datapath** composes the central processing unit (CPU/microprocessor).



Machine code

```
1111100010001001
111111111000010101100110
0000001001111110
1100001111110011
1101100011110111
1111101011101011
```

Computer h/w operates on a series of these machine code (binary) instructions through pulsating voltage signals that signify OFF or ON based on the binary digits 0 and 1 respectively.

Source: Page 247

Computer Organization and Design,
5th Edition, by David A. Patterson
and John L. Hennessy

Instruction Set Architecture (ISA) defines ...

1. Memory Model

- Word size
- Memory size $\rightarrow 2^m \times n$
 - 2^m distinct addressable locations in memory
 - each of these addressable locations has n bits
- Endianness
- Alignment
- Memory addressing modes

Instruction Set Architecture (ISA) defines ...

2. Registers

- Number
- Size
- Data type
- Purpose

Instruction Set Architecture (ISA) defines ...

machine code
instructions
and
corresponding
assembly
instructions

3. **Instruction Set** – all operations provided by ISA

➤ Description (semantic)

➤ For example, **sub*** means a subtraction operation

➤ Format

➤ For example, length, # and order of fields

opcode rs rt rd shamt func

➤ Operand model: number, order and meaning of operands

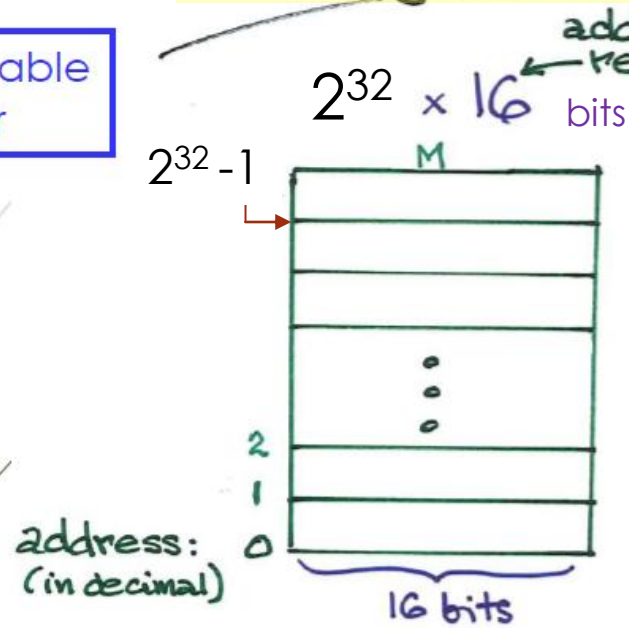
Instruction Set Architecture (ISA) cont'd

4. Model of computation

- Microprocessor executes our C program in such a way that it produces the expected result
 - We get the illusion that the microprocessor executes each C statement sequentially

Memory Models:

word-addressable computer



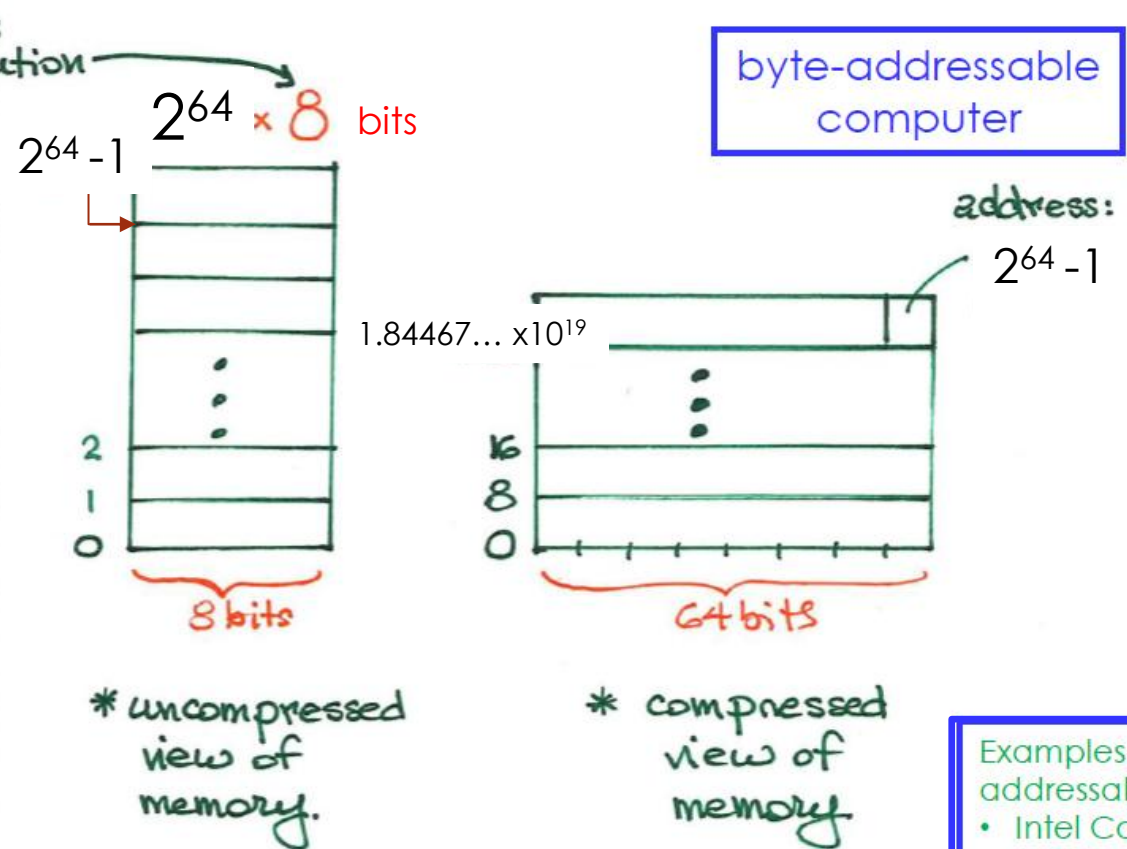
Examples of word-addressable ISAs:

- Data General Nova minicomputer
- Texas Instruments TMS9900
- National Semiconductor IMP-16

word size: 16 bits

address resolution: smallest addressable memory "chunk"

byte-addressable computer



word size: 64 bits.

Examples of byte-addressable ISAs:

- Intel Core i7 and i9
- AMD64 Epyc
- MIPS64
- RISC-V

Example of an ISA: x86

1. Memory model

- Word size: 64 bits
- Memory size $\rightarrow 2^m \times n$
 - m = 64 bits even though only 48 bits are used
 - n = 8 bits (byte-addressable)
- Memory addressing modes
 - Supports various addressing modes including immediate (direct), indirect, base+displacement, indexed, and scaled

2. Registers

- 16 integers registers of 64 bits (sections of 8/16/32/64 bits can be accessed)
 - **Purpose**: stack pointer, return value, PC, “general purpose” registers
- 16 floating point registers of 128 bits

3. Instruction set

- Lots of them: https://en.wikipedia.org/wiki/X86_instruction_listings
- Operand model: two operands (of different sizes)

Summary

- **Assembler** (part of the compilation process):
 - Transforms assembly code (`movl %edi, %eax`) into machine code (`0xf889 -> 1111100010001001`)
 - Machine code -> series of 0's and 1's
 - Control the microprocessor h/w
- **Instruction Set Architecture (ISA)**
 - **Definition:** Interface between the hardware and the software, specifying both what the microprocessor is capable of doing as well as how it gets done.
 - **Defines:**
 1. Memory model
 2. Registers
 3. Set of instructions (assembly-machine)
 4. Model of computationetc...

Next Lecture

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Example of an instruction set: MIPS
 - Create our own instruction sets
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Staged execution of machine instructions
 - Pipelined execution of machine instructions + Hazards